Assignment 2: CNN Theory Solutions

Output-size rule

For one spatial dimension, O = floor((N + 2P - K) / S) + 1, where N is the input size, P is padding per side, K is kernel size, and S is stride. These questions use dilation 1. A convolution produces one output channel per filter; pooling preserves the channel count.

Question 1

Input: 32 x 32 x 3; 8 filters, kernel 5 x 5, stride 1, padding 0.

O = floor((32 + 0 - 5) / 1) + 1 = 28.

Output: **28 x 28 x 8** (height x width x channels). Each filter spans all three input channels.

Question 2

Keep Question 1 settings but use same padding.

For a 5 x 5 kernel and stride 1, P = (5 - 1) / 2 = 2. O = floor((32 + 4 - 5) / 1) + 1 = 32.

Output: **32 x 32 x 8**. Same padding preserves the spatial dimensions for stride 1.

Question 3

Input: 64 x 64; kernel 3 x 3, stride 2, padding 0.

O = floor((64 - 3) / 2) + 1 = floor(30.5) + 1 = 31.

Output spatial size: **31 x 31**. Only fully contained windows are counted.

Question 4

Input feature map: 16 x 16; max pooling 2 x 2, stride 2.

O = floor((16 - 2) / 2) + 1 = 8.

Output: **8 x 8** per channel; pooling does not change the channel count.

Question 5

Input: 128 x 128; two convolutions, each kernel 3 x 3, stride 1, same padding.

Each layer uses P = 1: O = floor((128 + 2 - 3) / 1) + 1 = 128. The second layer gives the same result.

Output spatial shape: **128 x 128**. The final channel count equals the number of filters in the second layer; the question does not specify it.

Question 6: Removing model.train()

A newly constructed PyTorch module defaults to training mode. If the model has never been placed in evaluation mode, removing model.train() does not change that initial mode.

However, model.eval() switches the model and its submodules to evaluation mode. If validation calls model.eval() and the next training epoch omits model.train(), the model remains in evaluation mode. Dropout then stops randomly dropping activations, and batch normalization uses its running statistics rather than training batch statistics and does not update its running statistics.

The specified Assignment 2 CNN uses only convolution, ReLU, max pooling, flattening and linear layers. It contains neither dropout nor batch normalization, so its forward computation is identical in train and eval modes. Consequently, removing this line alone does not change this exact architecture's outputs or training behavior.

model.eval() does not disable gradients and does not prevent optimizer updates. Gradient computation is controlled separately, for example by torch.no_grad() or torch.inference_mode(). Calling model.train() before each training epoch is still good practice, particularly after validation or when adding mode-sensitive layers.

Architecture shape check

Stage	Output shape (excluding batch)
Input	3 x 64 x 64
Conv 1 + ReLU	16 x 64 x 64
Max pool 1	16 x 32 x 32
Conv 2 + ReLU	32 x 32 x 32
Max pool 2	32 x 16 x 16
Flatten	8192
Linear + ReLU	100
Final linear (logits)	10

Parameter count: (3 x 3 x 3 + 1) x 16 + (3 x 3 x 16 + 1) x 32 + (8192 + 1) x 100 + (100 + 1) x 10 = **825,398**.

Source: supplied Assignment2.pdf. Prepared with OpenAI Codex assistance.